

Модуль шифрования.

Создано системой Doxygen 1.9.4

1 Иерархический список классов	1
1.1 Иерархия классов	1
2 Алфавитный указатель классов	1
2.1 Классы	1
3 Список файлов	2
3.1 Файлы	2
4 Классы	2
4.1 Класс cipher_error	2
4.1.1 Подробное описание	3
4.1.2 Конструктор(ы)	3
4.2 Класс modAlphaCipher	4
4.2.1 Подробное описание	5
4.2.2 Конструктор(ы)	5
4.2.3 Методы	6
5 Файлы	10
5.1 Файл main.cpp	10
5.1.1 Подробное описание	11
5.1.2 Функции	11
5.2 Файл modAlphaCipher.cpp	12
5.2.1 Подробное описание	12
5.3 Файл modAlphaCipher.h	12
5.3.1 Подробное описание	13
5.4 modAlphaCipher.h	14
Предметный указатель	15

1 Иерархический список классов

1.1 Иерархия классов

Иерархия классов.

std::invalid_argument	
cipher_error	2
modAlphaCipher	4

2 Алфавитный указатель классов

2.1 Классы

Классы с их кратким описанием.

cipher_error	Класс исключения для ошибок шифрования	2
modAlphaCipher	Класс для шифрования методом Гронсфельда	4

3 Список файлов

3.1 Файлы

Полный список документированных файлов.

main.cpp	Главный модуль программы шифрования Гронсфельда	10
modAlphaCipher.cpp	Реализация модуля шифрования Гронсфельда	12
modAlphaCipher.h	Заголовочный файл для модуля шифрования Гронсфельда	12

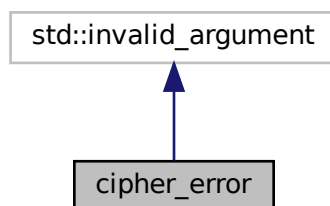
4 Классы

4.1 Класс cipher_error

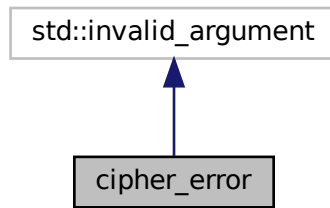
Класс исключения для ошибок шифрования

```
#include <modAlphaCipher.h>
```

Граф наследования: cipher_error:



Граф связей класса cipher_error:



Открытые члены

- `cipher_error` (`const std::string &what_arg`)
Конструктор с `std::string` в качестве параметра
- `cipher_error` (`const char *what_arg`)
Конструктор с C-строкой в качестве параметра

4.1.1 Подробное описание

Класс исключения для ошибок шифрования

Наследуется от `std::invalid_argument`, используется для обработки ошибок в процессе шифрования/дешифрования

4.1.2 Конструктор(ы)

4.1.2.1 `cipher_error()` [1/2] `cipher_error::cipher_error (`
`const std::string & what_arg )` [inline], [explicit]

Конструктор с `std::string` в качестве параметра

Аргументы

in	what_arg	Сообщение об ошибке
----	----------	---------------------

4.1.2.2 `cipher_error()` [2/2] `cipher_error::cipher_error (`
`const char * what_arg )` [inline], [explicit]

Конструктор с C-строкой в качестве параметра

Аргументы

in	what_arg	Сообщение об ошибке (C-строка)
----	----------	--------------------------------

Объявления и описания членов класса находятся в файле:

- [modAlphaCipher.h](#)

4.2 Класс modAlphaCipher

Класс для шифрования методом Гронсфельда

```
#include <modAlphaCipher.h>
```

Открытые члены

- [modAlphaCipher](#) ()=delete
Удаленный конструктор по умолчанию
- [modAlphaCipher](#) (const std::wstring &key)
Конструктор с ключом шифрования
- std::wstring [encrypt](#) (const std::wstring &open_text)
Шифрование открытого текста
- std::wstring [decrypt](#) (const std::wstring &cipher_text)
Дешифрование зашифрованного текста

Закрытые члены

- std::vector< int > [convert](#) (const std::wstring &s)
Преобразование строки в числовой вектор
- std::wstring [convert](#) (const std::vector< int > &v)
Преобразование числового вектора в строку
- std::wstring [getValidKey](#) (const std::wstring &s)
Валидация и нормализация ключа шифрования
- std::wstring [getValidOpenText](#) (const std::wstring &s)
Валидация открытого текста
- std::wstring [getValidCipherText](#) (const std::wstring &s)
Валидация зашифрованного текста

Закрытые данные

- std::wstring numAlpha = L"АБВГДЕЁЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ"
Алфавит для шифрования (33 буквы)
- std::map< wchar_t, int > alphaNum
Ассоциативный массив "символ -> индекс в алфавите".
- std::vector< int > key
Ключ шифрования в числовом виде

4.2.1 Подробное описание

Класс для шифрования методом Гронсфельда

Реализует шифрование и дешифрование текста на русском языке с использованием ключа-строки. Поддерживает русский алфавит (33 буквы). Алгоритм: каждая буква текста сдвигается на позицию соответствующей буквы ключа по модулю 33.

4.2.2 Конструктор(ы)

4.2.2.1 modAlphaCipher() [1/2] modAlphaCipher::modAlphaCipher () [delete]

Удаленный конструктор по умолчанию

Запрещено создание объекта без ключа

4.2.2.2 modAlphaCipher() [2/2] modAlphaCipher::modAlphaCipher (const std::wstring & skey)

Конструктор с ключом шифрования

Конструктор класса [modAlphaCipher](#).

Аргументы

in	skey	Ключ шифрования (строка русских букв)
----	------	---------------------------------------

Исключения

cipher_error	если ключ невалидный
------------------------------	----------------------

Заметки

Ключ будет преобразован к верхнему регистру и провалидирован

Аргументы

in	skey	Ключ шифрования
----	------	-----------------

Исключения

cipher_error	если ключ невалидный
------------------------------	----------------------

Инициализирует алфавит и преобразует ключ в числовой вид

4.2.3 Методы

4.2.3.1 `convert()` [1/2] `std::wstring modAlphaCipher::convert (`
`const std::vector< int > & v )` [private]

Преобразование числового вектора в строку

Аргументы

in	v	Вектор индексов символов
----	---	--------------------------

Возвращает

Строка символов

4.2.3.2 `convert()` [2/2] `std::vector< int > modAlphaCipher::convert (`
`const std::wstring & s )` [private]

Преобразование строки в числовой вектор

Аргументы

in	s	Входная строка
----	---	----------------

Возвращает

Вектор индексов символов в алфавите

Заметки

Пробелы игнорируются

Аргументы

in	s	Входная строка
----	---	----------------

Возвращает

Вектор индексов символов

Заметки

Пробелы игнорируются

4.2.3.3 `decrypt()` `std::wstring modAlphaCipher::decrypt (`
`const std::wstring & cipher_text )`

Дешифрование зашифрованного текста

Аргументы

in	cipher_text	Зашифрованный текст
----	-------------	---------------------

Возвращает

Расшифрованный текст

Исключения

<code>cipher_error</code>	если шифротекст невалидный
---------------------------	----------------------------

Аргументы

in	cipher_text	Зашифрованный текст
----	-------------	---------------------

Возвращает

Расшифрованный текст

Исключения

<code>cipher_error</code>	если шифротекст невалидный
---------------------------	----------------------------

Выполняется обратный сдвиг букв текста

4.2.3.4 `encrypt()` `std::wstring modAlphaCipher::encrypt (`
`const std::wstring & open_text )`

Шифрование открытого текста

Аргументы

in	open_text	Открытый текст для шифрования
----	-----------	-------------------------------

Возвращает

Зашифрованный текст

Исключения

<code>cipher_error</code>	если текст невалидный
---------------------------	-----------------------

Строчные буквы преобразуются в прописные, не-буквы удаляются

Аргументы

in	open_text	Открытый текст
----	-----------	----------------

Возвращает

Зашифрованный текст

Исключения

<code>cipher_error</code>	если текст невалидный
---------------------------	-----------------------

Каждая буква текста сдвигается на позицию соответствующей буквы ключа

```
4.2.3.5  getValidCipherText()  std::wstring modAlphaCipher::getValidCipherText (
                                     const std::wstring & s )  [private]
```

Валидация зашифрованного текста

Аргументы

in	s	Зашифрованный текст
----	---	---------------------

Возвращает

Валидный шифротекст

Исключения

<code>cipher_error</code>	если текст пустой, содержит пробелы или недопустимые символы
---------------------------	--

Аргументы

in	s	Зашифрованный текст
----	---	---------------------

Возвращает

Валидный шифротекст

Исключения

<code>cipher_error</code>	если шифротекст невалидный
---------------------------	----------------------------

4.2.3.6 `getValidKey()` `std::wstring modAlphaCipher::getValidKey (`
`const std::wstring & s ) [private]`

Валидация и нормализация ключа шифрования

Валидация ключа шифрования

Аргументы

in	s	Исходный ключ
----	---	---------------

Возвращает

Валидный ключ (в верхнем регистре, только русские буквы)

Исключения

<code>cipher_error</code>	если ключ пустой, содержит пробелы, цифры, знаки пунктуации или не русские буквы
---------------------------	--

Аргументы

in	s	Исходный ключ
----	---	---------------

Возвращает

Валидный ключ

Исключения

<code>cipher_error</code>	если ключ невалидный
---------------------------	----------------------

4.2.3.7 `getValidOpenText()` `std::wstring modAlphaCipher::getValidOpenText (`
`const std::wstring & s ) [private]`

Валидация открытого текста

Аргументы

in	s	Исходный открытый текст
----	---	-------------------------

Возвращает

Валидный текст (русские буквы в верхнем регистре, пробелы сохраняются)

Исключения

cipher_error	если текст пустой, не содержит русских букв или содержит недопустимые символы
------------------------------	---

Аргументы

in	s	Исходный открытый текст
----	---	-------------------------

Возвращает

Валидный текст

Исключения

cipher_error	если текст невалидный
------------------------------	-----------------------

Объявления и описания членов классов находятся в файлах:

- [modAlphaCipher.h](#)
- [modAlphaCipher.cpp](#)

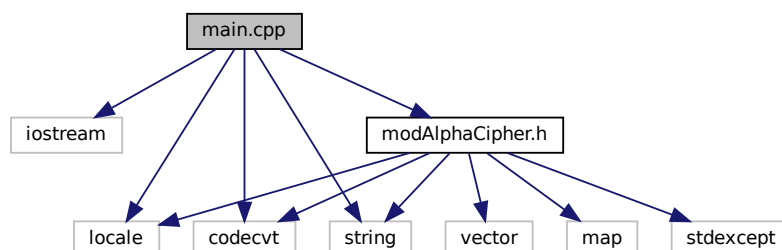
5 Файлы

5.1 Файл main.cpp

Главный модуль программы шифрования Гронсфельда

```
#include <iostream>
#include <locale>
#include <codecvt>
#include <string>
#include "modAlphaCipher.h"
```

Граф включаемых заголовочных файлов для main.cpp:



Функции

- `int main ()`

Главная функция программы

5.1.1 Подробное описание

Главный модуль программы шифрования Гронсфельда

Содержит пользовательский интерфейс для работы с шифром Гронсфельда

Автор

Клименко Глеб

Дата

01.12.2025

Версия

1.0

5.1.2 Функции

5.1.2.1 `main()` `int main ()`

Главная функция программы

Возвращает

0 при успешном завершении, 1 при ошибке

Обеспечивает взаимодействие с пользователем:

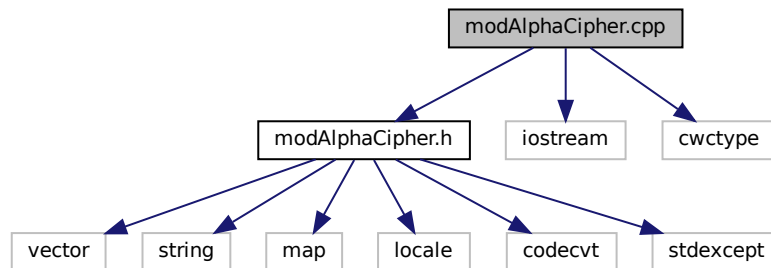
- Ввод ключа шифрования
- Ввод текста для шифрования
- Шифрование и вывод результата
- Дешифрование и проверка результата

5.2 Файл modAlphaCipher.cpp

Реализация модуля шифрования Гронсфельда

```
#include "modAlphaCipher.h"  
#include <iostream>  
#include <cwctype>
```

Граф включаемых заголовочных файлов для modAlphaCipher.cpp:



5.2.1 Подробное описание

Реализация модуля шифрования Гронсфельда

Содержит реализации методов класса [modAlphaCipher](#)

Автор

Клименко Глеб

Дата

01.12.2025

Версия

1.0

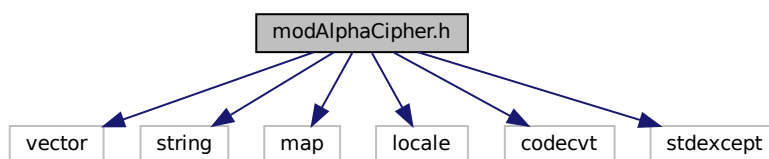
5.3 Файл modAlphaCipher.h

Заголовочный файл для модуля шифрования Гронсфельда

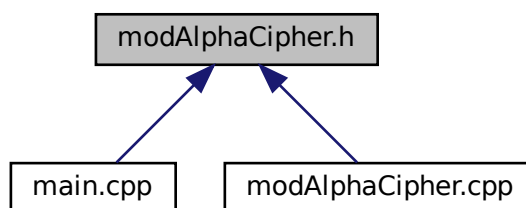
```
#include <vector>  
#include <string>  
#include <map>  
#include <locale>  
#include <codecvt>
```

```
#include <stdexcept>
```

Граф включаемых заголовочных файлов для modAlphaCipher.h:



Граф файлов, в которые включается этот файл:



Классы

- class `cipher_error`
Класс исключения для ошибок шифрования
- class `modAlphaCipher`
Класс для шифрования методом Гронсфельда

5.3.1 Подробное описание

Заголовочный файл для модуля шифрования Гронсфельда

Содержит объявления классов для шифрования и дешифрования текста методом Гронсфельда на русском языке

Автор

Клименко Глеб

Дата

01.12.2025

Версия

1.0

Авторство

Учебный проект

Предупреждения

Реализация только для русского языка

5.4 modAlphaCipher.h

[См. документацию.](#)

```
1
13 #pragma once
14 #include <vector>
15 #include <string>
16 #include <map>
17 #include <locale>
18 #include <codecvt>
19 #include <stdexcept>
20
27 class cipher_error : public std::invalid_argument {
28 public:
33     explicit cipher_error(const std::string& what_arg) :
34         std::invalid_argument(what_arg) {}
35
40     explicit cipher_error(const char* what_arg) :
41         std::invalid_argument(what_arg) {}
42 };
43
52 class modAlphaCipher
53 {
54 private:
55     std::wstring numAlpha = L"АБВГДЕЁЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ";
56     std::map<wchar_t, int> alphaNum;
57     std::vector<int> key;
58
65     std::vector<int> convert(const std::wstring& s);
66
72     std::wstring convert(const std::vector<int>& v);
73
81     std::wstring getValidKey(const std::wstring& s);
82
90     std::wstring getValidOpenText(const std::wstring& s);
91
99     std::wstring getValidCipherText(const std::wstring& s);
100
101 public:
106     modAlphaCipher() = delete;
107
114     modAlphaCipher(const std::wstring& skey);
115
123     std::wstring encrypt(const std::wstring& open_text);
124
131     std::wstring decrypt(const std::wstring& cipher_text);
132 };
```

Предметный указатель

- cipher_error, [2](#)
 - cipher_error, [3](#)
- convert
 - modAlphaCipher, [6](#)
- decrypt
 - modAlphaCipher, [6](#)
- encrypt
 - modAlphaCipher, [7](#)
- getValidCipherText
 - modAlphaCipher, [8](#)
- getValidKey
 - modAlphaCipher, [8](#)
- getValidOpenText
 - modAlphaCipher, [9](#)
- main
 - main.cpp, [11](#)
- main.cpp, [10](#)
 - main, [11](#)
- modAlphaCipher, [4](#)
 - convert, [6](#)
 - decrypt, [6](#)
 - encrypt, [7](#)
 - getValidCipherText, [8](#)
 - getValidKey, [8](#)
 - getValidOpenText, [9](#)
 - modAlphaCipher, [5](#)
- modAlphaCipher.cpp, [12](#)
- modAlphaCipher.h, [12](#)